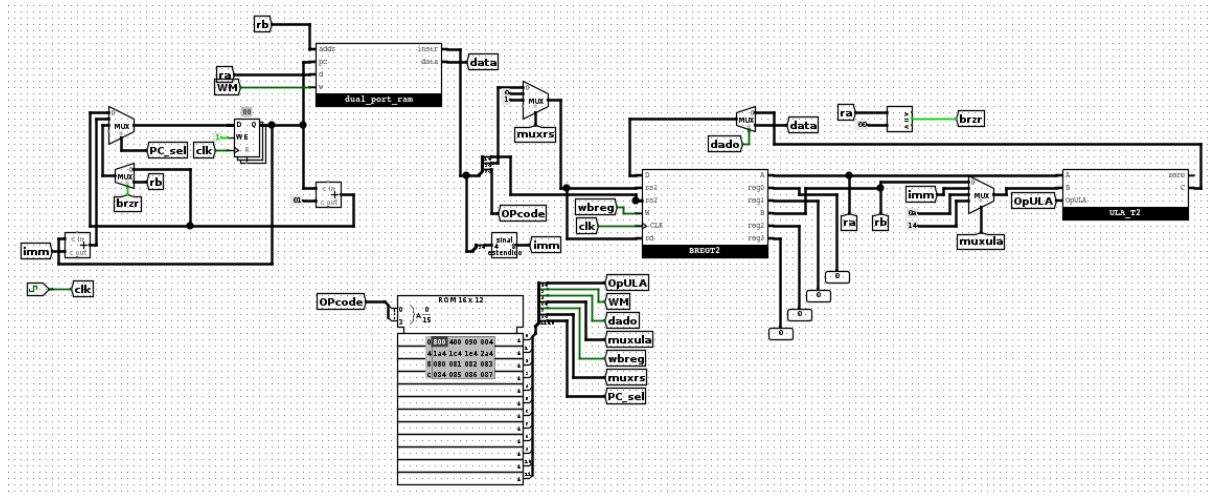


RELATÓRIO T2
ALUNO: FABIO ALEIXO DE OLIVEIRA KARVAT
GRR20254246



A implementação do processador foi monociclo.

CÓDIGOS DA ULA:

not	000
and	001
or	010
xor	011
add	100
sub	101
slr	110
srr	111

TABELA DE CONTROLE:

instrução	pc_sel	muxrs	wbreg	muxula	dado	WM	OPULA
brzr	10	00	0	00	0	0	000
ji	01	00	0	00	0	0	000
ld	00	00	1	00	1	0	000
st	00	00	0	00	0	1	000
addi	00	01	1	01	0	0	100
addi10	00	01	1	10	0	0	100
addi20	00	01	1	11	0	0	100
addir1	00	10	1	01	0	0	100
not	00	0	1	00	0	0	000
and	00	0	1	00	0	0	001
or	00	0	1	00	0	0	010
xor	00	0	1	00	0	0	011
add	00	0	1	00	0	0	100
sub	00	0	1	00	0	0	101
slr	00	0	1	00	0	0	110
srr	00	0	1	00	0	0	111

ASSEMBLY DO PROGRAMA:

```
addir1 7  
addir1 3
```

```
addir1 -1  
addi20  
addi20
```

```
ji 2  
ji -4
```

```
add r0, r1  
ld r2, r0  
add r3, r2  
sub r0, r0
```

```
ji 2  
ji -6
```

```
addi20  
addi20  
addi10
```

```
ji 2  
ji -5
```

```
add r0, r1  
ld r2, r0  
add r3, r2  
sub r0, r0
```

```
ji 2  
ji -6
```

```
addi20  
addi20  
addi20
```

```
ji 2  
ji -5
```

```
add r0, r1  
st r3, r0  
sub r3, r3  
sub r0, r0
```

```
brzr r1, r1  
ji -6
```

A: está no endereço 0x28
0 2 4 6 8 10 12 14 16 18

B: está no endereço 0x32
1 3 5 7 9 11 13 15 17 19

R: está no endereço 0x3c
0 0 0 0 0 0 0 0 0 0

INSTRUÇÕES ADICIONADAS:

```
addi10 -> r0 = r0 + 10  
addi20 -> r0 = r0 + 20  
addir1 -> r1 = r1 + imm
```

justificativa:

addi10 e addi20 agilizam para carregar valores altos como os endereços de memória.

addir1 carrega direto no r1, assim, não é preciso carregar no r0 e depois passar pro r1 o que encurtou meu código.